

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Основы системного программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Использование динамических библиотек»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Грозов Владимир Андреевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ЗАДАНИЕ. ВАРИАНТ 19.....	3
ХОД РАБОТЫ.....	4

ЗАДАНИЕ. ВАРИАНТ 19

Таблица 1: Интерфейсные функции библиотеки-плагина

Функция	Назначение
plugin_get_info	Получение информации о поддерживаемых плагином опциях.
plugin_process_file	Проверка соответствия файла заданным критериям.

Таблица 2: Опции командной строки, поддерживаемые программой

Опция	Назначение
-P dir	Каталог с плагинами.
-A	Объединение опций плагинов с помощью операции «И» (действует по умолчанию).
-O	Объединение опций плагинов с помощью операции «ИЛИ».
-N	Инвертирование условия поиска (после объединения опций плагинов с помощью -A или -O).
-v	Вывод версии программы и информации о программе (ФИО -v исполнителя, номер группы, номер варианта лабораторной).
-h	Вывод справки по опциям.

Таблица 3: Переменные окружения, поддерживаемые программой и динамической библиотекой

Переменная	Назначение
LAB2DEBUG	Включение вывода отладочной информации

Таблица 4: Опции, поддерживаемые библиотекой-плагином (вариант 19)

Вариант (номер)	Описание поддерживаемых плагином опций
19	Опция: --ipv4-addr-bin <значение> Назначение: поиск файлов, содержащих заданное значение IPv4-адреса в бинарной (little-endian или big-endian) форме, т. е. в виде четырех последовательных октетов с заданными значениями. Пример: --ipv4-addr-bin 192.168.8.1

ХОД РАБОТЫ

Исходный код Makefile:

```
1 | CC = gcc
2 | CFLAGS = -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE
3 | LDFLAGS = -ldl
4 | TARGET = lab2vypN3250
5 | LIB = libvypN3250.so
6 |
7 | all: $(TARGET) $(LIB)
8 |
9 | $(TARGET): lab2vypN3250.c
10 |     $(CC) $(CFLAGS) lab2vypN3250.c -o $(TARGET) $(LDFLAGS)
11 |
12 | $(LIB): libvypN3250.c
13 |     $(CC) $(CFLAGS) -fPIC -shared libvypN3250.c -o $(LIB)
14 |
15 | clean:
16 |     rm -f $(TARGET) $(LIB) *.o
17 |     rm -rf test
18 |
19 |
20 | EXE = ./lab2vypN3250
21 | TEST_DIR = test
22 | PLUGIN_PATH = .
23 |
24 | test: all
25 |     @echo ""
26 |     @echo "--- Подготовка среды для тестирования опций -A -O -N ---"
27 |     rm -rf $(TEST_DIR)
28 |     mkdir -p $(TEST_DIR)
29 |     # Файл 1: Содержит только IP 1.1.1.1
30 |     printf "\x01\x01\x01\x01" > $(TEST_DIR)/only_1.bin
31 |     # Файл 2: Содержит только IP 2.2.2.2
32 |     printf "\x02\x02\x02\x02" > $(TEST_DIR)/only_2.bin
33 |     # Файл 3: Содержит оба IP
34 |     printf "\x01\x01\x01\x01\x00\x00\x02\x02\x02\x02" > $(TEST_DIR)/both.bin
35 |     # Файл 4: Пустой файл (не содержит ничего)
36 |     touch $(TEST_DIR)/empty.txt
37 |     @echo ""
38 |
39 |     @echo "---- Тест -A ----"
40 |     $(EXE) -P $(PLUGIN_PATH) -A --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 $(TEST_DIR)
41 |     @echo ""
42 |
43 |     @echo "---- Тест -O ----"
44 |     $(EXE) -P $(PLUGIN_PATH) -O --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 $(TEST_DIR)
45 |     @echo ""
46 |
47 |     @echo "---- Тест -N ----"
48 |     $(EXE) -P $(PLUGIN_PATH) -N --ipv4-addr-bin 1.1.1.1 $(TEST_DIR)
49 |     @echo ""
50 |
51 |     @echo "---- Тест -N -O ----"
52 |     $(EXE) -P $(PLUGIN_PATH) -N -O --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 $(TEST_DIR)
53 |     @echo ""
54 |
55 |
56 | .PHONY: test
```

Отчет valgrind со строки HEAP SUMMARY:

```
==35158== HEAP SUMMARY:
==35158==      in use at exit: 0 bytes in 0 blocks
==35158==    total heap usage: 25 allocs, 25 frees, 83,737 bytes allocated
==35158==
==35158== All heap blocks were freed -- no leaks are possible
==35158==
==35158== For lists of detected and suppressed errors, rerun with: -s
==35158== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```

vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -v
Программа lab2vypN3250
Пахомов Владимир Юрьевич, гр. N3250
Вариант: 19
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -h
Использование: lab2vypN3250 [опции] <каталог>
Опции:
  -v, --version  Вывод информации об авторе
  -h, --help     Вывод этой справки
  -A             Логическое И
  -O             Логическое OR
  -N             Логическое NOT
  -P <каталог>  Указать каталог с плагинами
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -j
./lab2vypN3250: invalid option -- 'j'
Введены неизвестные опции. Используйте -h для справки
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 --ipv4-addr-bin 1.1.1.1 --
ipv4-addr-bin 2.2.2.2 /home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/both.bin
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -P . --ipv4-addr-bin 1.1.1.1
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_1.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/both.bin
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -P . -N --ipv4-addr-bin 1.1.
1.1 /home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_2.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/empty.txt
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 ./lab2vypN3250 -O --ipv4-addr-bin 1.1.1.1 -
-ipv4-addr-bin 2.2.2.2 /home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_1.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_2.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/both.bin
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 LAB2DEBUG=1 ./lab2vypN3250 -O --ipv4-addr-b
in 1.1.1.1 --ipv4-addr-bin 2.2.2.2 /home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test

[DEBUG] Плагин libvypN3250: Найден IP 1.1.1.1 (big-endian) по смещению 0 в файле /home/vladimir/De
sktop/ОСП/ЛР2/lab2vypN3250/test/only_1.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_1.bin

[DEBUG] Плагин libvypN3250: Найден IP 2.2.2.2 (big-endian) по смещению 0 в файле /home/vladimir/De
sktop/ОСП/ЛР2/lab2vypN3250/test/only_2.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/only_2.bin

[DEBUG] Плагин libvypN3250: Найден IP 1.1.1.1 (big-endian) по смещению 0 в файле /home/vladimir/De
sktop/ОСП/ЛР2/lab2vypN3250/test/both.bin

[DEBUG] Плагин libvypN3250: Найден IP 2.2.2.2 (big-endian) по смещению 6 в файле /home/vladimir/De
sktop/ОСП/ЛР2/lab2vypN3250/test/both.bin
/home/vladimir/Desktop/ОСП/ЛР2/lab2vypN3250/test/both.bin
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250

```

Рисунок 1: Пример работы программы

```
vladimir@vladimirp ~/Desktop/ОСП/ЛР2/lab2vypN3250 make test

--- Подготовка среды для тестирования опций -A -O -N ---
rm -rf test
mkdir -p test
# Файл 1: Содержит только IP 1.1.1.1
printf "\x01\x01\x01\x01" > test/only_1.bin
# Файл 2: Содержит только IP 2.2.2.2
printf "\x02\x02\x02\x02" > test/only_2.bin
# Файл 3: Содержит оба IP
printf "\x01\x01\x01\x01\x00\x00\x02\x02\x02\x02" > test/both.bin
# Файл 4: Пустой файл (не содержит ничего)
touch test/empty.txt

---- Тест -A ----
./lab2vypN3250 -P . -A --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 test
test/both.bin

---- Тест -O ----
./lab2vypN3250 -P . -O --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 test
test/only_1.bin
test/only_2.bin
test/both.bin

---- Тест -N ----
./lab2vypN3250 -P . -N --ipv4-addr-bin 1.1.1.1 test
test/only_2.bin
test/empty.txt

---- Тест -N -O ----
./lab2vypN3250 -P . -N -O --ipv4-addr-bin 1.1.1.1 --ipv4-addr-bin 2.2.2.2 test
test/empty.txt
```

Рисунок 2: Результат работы тестов из Makefile

Исходный код программы lab2vypN3250:

```
1  | #include <stdio.h>
2  | #include <stdlib.h>
3  | #include <string.h>
4  | #include <dirent.h>
5  | #include <dlfcn.h>
6  | #include <errno.h>
7  | #include <getopt.h>
8  | #include <fcntl.h>
9  | #include <unistd.h>
10 | #include <sys/syscall.h>
11 | #include <libgen.h>
12 | #include <stdint.h>
13 | #include <sys/stat.h>
14 | #include "plugin_api.h"
15 |
16 | #define INITIAL_BUFFER_SIZE 8192
17 |
18 | typedef struct option_group {
19 |     struct option *opts;
20 |     size_t count;
21 | } option_group_t;
22 |
23 | typedef struct loaded_plugin {
24 |     void *handle;
25 |     struct plugin_info info;
26 |     int (*process_file)(const char *fname, struct option in_opts[], size_t in_opts_len);
27 |
28 |     option_group_t *groups;
29 |     size_t groups_count;
30 | } loaded_plugin_t;
31 |
32 | typedef struct StackNode {
33 |     char *path;
34 |     struct StackNode *next;
35 | } StackNode;
36 |
37 | void push(StackNode **top, const char *path) {
38 |     StackNode *node = malloc(sizeof(StackNode));
39 |     if (!node) exit(1);
40 |     node->path = strdup(path);
41 |     node->next = *top;
42 |     *top = node;
43 | }
44 |
45 | char *pop(StackNode **top) {
46 |     if (*top == NULL) return NULL;
47 |     StackNode *node = *top;
48 |     char *path = node->path;
49 |     *top = node->next;
50 |     free(node);
51 |     return path;
52 | }
53 |
54 | struct linux_dirent64 {
55 |     unsigned long long d_ino;
56 |     long long d_off;
57 |     unsigned short d_reclen;
58 |     unsigned char d_type;
59 |     char d_name[];
60 | };
61 |
62 | void deinit_all(loaded_plugin_t *plugins, size_t count, char *p_dir, struct option *l_opts) {
63 |     for (size_t i = 0; i < count; i++) {
64 |         for (size_t g = 0; g < plugins[i].groups_count; g++) {
65 |             for (size_t j = 0; j < plugins[i].groups[g].count; j++) {
66 |                 if (plugins[i].groups[g].opts[j].flag) {
67 |                     free(plugins[i].groups[g].opts[j].flag);
68 |                 }
69 |             }
70 |             free(plugins[i].groups[g].opts);
71 |         }
72 |     }
73 | }
```

```

72         free(plugin[i].groups);
73         dlclose(plugin[i].handle);
74     }
75     if (plugins) free(plugins);
76     if (p_dir) free(p_dir);
77     if (l_opts) free(l_opts);
78 }
79
80 int main(int argc, char *argv[]) {
81     int op_and = 1, op_not = 0;
82     char *plugin_dir = NULL;
83
84     char *exe_path = malloc(4096);
85     ssize_t rlen = readlink("/proc/self/exe", exe_path, 4095);
86     if (rlen != -1) {
87         exe_path[rlen] = '\0';
88         plugin_dir = strdup(dirname(exe_path));
89     } else {
90         plugin_dir = strdup(".");
91     }
92     free(exe_path);
93
94     loaded_plugin_t *plugins = NULL;
95     size_t plugins_count = 0;
96
97     for (int i = 1; i < argc; i++) {
98         if (strcmp(argv[i], "-p") == 0 && i + 1 < argc) {
99             free(plugin_dir);
100             plugin_dir = strdup(argv[i+1]);
101         }
102     }
103
104     DIR *d = opendir(plugin_dir);
105     if (d) {
106         struct dirent *dir;
107         while ((dir = readdir(d)) != NULL) {
108             if (strstr(dir->d_name, ".so")) {
109                 char *full_p = malloc(strlen(plugin_dir) + strlen(dir->d_name) + 2);
110                 sprintf(full_p, "%s/%s", plugin_dir, dir->d_name);
111                 void *h = dlopen(full_p, RTLD_LAZY);
112                 free(full_p);
113                 if (h) {
114                     int (*gi)(struct plugin_info*) = dlsym(h, "plugin_get_info");
115                     int (*pf)(const char*, struct option[], size_t) = dlsym(h,
116 "plugin_process_file");
117                     if (gi && pf) {
118                         plugins = realloc(plugins, sizeof(loaded_plugin_t) * (plugins_count +
119 1));
120                         plugins[plugins_count].handle = h;
121                         plugins[plugins_count].process_file = pf;
122                         plugins[plugins_count].groups = NULL;
123                         plugins[plugins_count].groups_count = 0;
124                         gi(&plugins[plugins_count].info);
125                         plugins_count++;
126                     } else { dlclose(h); }
127                 }
128             }
129         }
130         closedir(d);
131     }
132
133     size_t total_opts = 3;
134     for(size_t i=0; i<plugins_count; i++) total_opts += plugins[i].info.sup_opts_len;
135     struct option *long_options = calloc(total_opts, sizeof(struct option));
136
137     int opt_idx = 0;
138     long_options[opt_idx++] = (struct option){"version", no_argument, 0, 'v'};
139     long_options[opt_idx++] = (struct option){"help", no_argument, 0, 'h'};
140     for (size_t i = 0; i < plugins_count; i++) {
141         for (size_t j = 0; j < plugins[i].info.sup_opts_len; j++)
142             long_options[opt_idx++] = plugins[i].info.sup_opts[j].opt;
143     }
144
145     int opt, l_idx = 0;

```

```

144 |         while ((opt = getopt_long(argc, argv, "P:AONvh", long_options, &l_idx)) != -1) {
145 |             if (opt == 'v') {
146 |                 printf("Программа lab2vypN3250\n");
147 |                 printf("Пахомов Владимир Юрьевич, гр. N3250\nВариант: 19\n");
148 |                 deinit_all(plugins, plugins_count, plugin_dir, long_options); return 0;
149 |             }
150 |             else if (opt == 'h') {
151 |                 printf("Использование: lab2vypN3250 [опции] <каталог>\n");
152 |                 printf("Опции:\n -v, --version Вывод информации об авторе\n -h, --help
Вывод этой справки\n");
153 |                 printf(" -A          Логическое И\n -O          Логическое OR\n -N
Логическое NOT\n -P <каталог> Указать каталог с плагинами\n");
154 |                 deinit_all(plugins, plugins_count, plugin_dir, long_options); return 0;
155 |             }
156 |             else if (opt == 'A') op_and = 1;
157 |             else if (opt == 'O') op_and = 0;
158 |             else if (opt == 'N') op_not = 1;
159 |             else if (opt == 'P') continue;
160 |             else if (opt == 0) {
161 |                 for (size_t i = 0; i < plugins_count; i++) {
162 |                     for (size_t j = 0; j < plugins[i].info.sup_opts_len; j++) {
163 |                         if (strcmp(long_options[l_idx].name,
plugins[i].info.sup_opts[j].opt.name) == 0) {
164 |
165 |                             int need_new_group = 0;
166 |                             if (plugins[i].groups_count > 0) {
167 |                                 size_t last_g = plugins[i].groups_count - 1;
168 |                                 for (size_t k = 0; k < plugins[i].groups[last_g].count; k++) {
169 |                                     if (strcmp(plugins[i].groups[last_g].opts[k].name,
long_options[l_idx].name) == 0) {
170 |                                         need_new_group = 1;
171 |                                         break;
172 |                                     }
173 |                                 }
174 |                             } else {
175 |                                 need_new_group = 1;
176 |                             }
177 |
178 |                             if (need_new_group) {
179 |                                 plugins[i].groups = realloc(plugins[i].groups,
sizeof(option_group_t) * (plugins[i].groups_count + 1));
180 |                                 plugins[i].groups[plugins[i].groups_count].opts = NULL;
181 |                                 plugins[i].groups[plugins[i].groups_count].count = 0;
182 |                                 plugins[i].groups_count++;
183 |                             }
184 |
185 |                             size_t cur_g = plugins[i].groups_count - 1;
186 |                             plugins[i].groups[cur_g].opts =
realloc(plugins[i].groups[cur_g].opts, sizeof(struct option) * (plugins[i].groups[cur_g].count + 1));
187 |
188 |                             struct option new_opt = plugins[i].info.sup_opts[j].opt;
189 |                             new_opt.flag = optarg ? (int*)strdup(optarg) : NULL;
190 |
191 |                             plugins[i].groups[cur_g].opts[plugins[i].groups[cur_g].count] =
new_opt;
192 |                             plugins[i].groups[cur_g].count++;
193 |                         }
194 |                     }
195 |                 }
196 |             } else {
197 |                 fprintf(stderr, "Введены неизвестные опции. Используйте -h для справки\n");
198 |                 deinit_all(plugins, plugins_count, plugin_dir, long_options); return 1;
199 |             }
200 |         }
201 |
202 |         char *start_dir = (optind < argc) ? argv[optind] : ".";
203 |         DIR *check_d = opendir(start_dir);
204 |         if (!check_d) {
205 |             fprintf(stderr, "Ошибка: '%s': %s\n", start_dir, strerror(errno));
206 |             deinit_all(plugins, plugins_count, plugin_dir, long_options); return 1;
207 |         }
208 |         closedir(check_d);
209 |
210 |         StackNode *stack = NULL;

```

```

211 |     push(&stack, start_dir);
212 |
213 |     while (stack) {
214 |         char *curr = pop(&stack);
215 |         int fd = open(curr, O_RDONLY | O_DIRECTORY);
216 |         if (fd == -1) { free(curr); continue; }
217 |
218 |         char *buf = malloc(INITIAL_BUFFER_SIZE);
219 |         long nread;
220 |         while ((nread = syscall(SYS_getdents64, fd, buf, INITIAL_BUFFER_SIZE)) > 0) {
221 |             for (long bpos = 0; bpos < nread; ) {
222 |                 struct linux_dirent64 *de = (struct linux_dirent64 *) (buf + bpos);
223 |                 if (strcmp(de->d_name, ".") == 0 || strcmp(de->d_name, "..") == 0) {
224 |                     bpos += de->d_reclen; continue;
225 |                 }
226 |
227 |                 char *fp = malloc(strlen(curr) + strlen(de->d_name) + 2);
228 |                 sprintf(fp, "%s/%s", curr, de->d_name);
229 |
230 |                 if (de->d_type == DT_DIR) {
231 |                     push(&stack, fp);
232 |                     free(fp);
233 |                 } else if (de->d_type == DT_REG) {
234 |                     int final = op_and;
235 |                     int run = 0;
236 |
237 |                     for (size_t i = 0; i < plugins_count; i++) {
238 |                         for (size_t g = 0; g < plugins[i].groups_count; g++) {
239 |                             run = 1;
240 |                             int res = (plugins[i].process_file(fp, plugins[i].groups[g].opts,
plugins[i].groups[g].count) == 0);
241 |
242 |                             if (op_and) final &= res;
243 |                             else final |= res;
244 |                         }
245 |                     }
246 |                     if (op_not) final = !final;
247 |                     if (final || !run) printf("%s\n", fp);
248 |                     free(fp);
249 |                 } else {
250 |                     free(fp);
251 |                 }
252 |                 bpos += de->d_reclen;
253 |             }
254 |         }
255 |         free(buf);
256 |         close(fd);
257 |         free(curr);
258 |     }
259 |
260 |     deinit_all(plugins, plugins_count, plugin_dir, long_options);
261 |     return 0;
262 | }

```

Исходный код программы libvypN3250.c:

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <fcntl.h>
5 | #include <unistd.h>
6 | #include <stdint.h>
7 | #include "plugin_api.h"
8 |
9 | static uint32_t reverseIP(uint32_t v) {
10 |     return ((v >> 24) & 0xFF) | ((v >> 8) & 0xFF00) |
11 |         ((v << 8) & 0xFF0000) | ((v << 24) & 0xFF000000);
12 | }
13 |
14 | static int parse_ipv4(const char *str, uint32_t *out_be) {
15 |     unsigned int b[4];

```

```

16     char tail[2];
17
18     int dots = 0;
19     for (const char *p = str; *p; p++) {
20         if (*p == '.') dots++;
21     }
22     if (dots != 3) return 0;
23
24     if (sscanf(str, "%u.%u.%u.%u%1s", &b[0], &b[1], &b[2], &b[3], tail) != 4) {
25         return 0;
26     }
27
28     for (int i = 0; i < 4; i++) {
29         if (b[i] > 255) return 0;
30     }
31
32     unsigned char *p = (unsigned char *)out_be;
33     p[0] = (unsigned char)b[0];
34     p[1] = (unsigned char)b[1];
35     p[2] = (unsigned char)b[2];
36     p[3] = (unsigned char)b[3];
37
38     return 1;
39 }
40
41 static struct plugin_option my_options[] = {
42     {
43         {"ipv4-addr-bin", required_argument, 0, 0},
44         "Поиск IPv4 адреса в бинарном виде (BE/LE)"
45     }
46 };
47
48 static struct plugin_info my_info = {
49     "Плагин поиска бинарной IPv4 строки",
50     "Пахомов Владимир Юрьевич, гр. N3250\nВариант: 19",
51     1,
52     my_options
53 };
54
55 int plugin_get_info(struct plugin_info* ppi) {
56     if (!ppi) return -1;
57     *ppi = my_info;
58     return 0;
59 }
60
61 int plugin_process_file(const char *fname, struct option in_opts[], size_t in_opts_len) {
62     int debug = (getenv("LAB2DEBUG") != NULL);
63
64     for (size_t i = 0; i < in_opts_len; i++) {
65         if (strcmp(in_opts[i].name, "ipv4-addr-bin") == 0) {
66             const char *ip_str = (const char *)in_opts[i].flag;
67             uint32_t be_val = 0;
68
69             if (!parse_ipv4(ip_str, &be_val)) {
70                 fprintf(stderr, "Ошибка: некорректный формат IPv4: %s\n", ip_str);
71                 return -1;
72             }
73
74             uint32_t le_val = reverseIP(be_val);
75             int fd = open(fname, O_RDONLY);
76             if (fd < 0) return 1;
77
78             unsigned char buf[4096];
79             ssize_t n;
80             off_t total_read = 0;
81             int found = 0;
82
83             while ((n = read(fd, buf, sizeof(buf))) > 0) {
84                 for (int j = 0; j <= (int)n - 4; j++) {
85                     uint32_t chunk;
86                     memcpy(&chunk, &buf[j], 4);
87                     if (chunk == be_val || chunk == le_val) {
88                         if (debug) {

```

```

89 |                                     fprintf(stderr, "\n[DEBUG] Плагин libvypN3250: Найден IP %s (%s)
по смещению %ld в файле %s\n",
90 |                                     ip_str, chunk == be_val ? "big-endian" : "litte-endian",
(long)(total_read + j), fname);
91 |                                     }
92 |                                     found = 1;
93 |                                     break;
94 |                                     }
95 |                                     }
96 |                                     if (found) break;
97 |
98 |                                     if (n == (ssize_t)sizeof(buf)) {
99 |                                         off_t back_step = 3;
100 |                                         lseek(fd, -back_step, SEEK_CUR);
101 |                                         total_read += (n - back_step);
102 |                                     } else {
103 |                                         total_read += n;
104 |                                     }
105 |                                     }
106 |                                     close(fd);
107 |
108 |                                     if (!found) return 1;
109 |                                 }
110 |                             }
111 |                             return 0;
112 |     }

```

Исходный код программы plugin_api.h:

```
1  | #ifndef _PLUGIN_API_H
2  | #define _PLUGIN_API_H
3  |
4  | #include <getopt.h>
5  | #include <stddef.h>
6  |
7  | #define PLUGIN_API_VERSION 1
8  |
9  | /*
10 |  * Структура, описывающая опцию, поддерживаемую плагином.
11 |  */
12 | struct plugin_option {
13 |     /* Опция в формате, поддерживаемом getopt_long (man 3 getopt_long). */
14 |     struct option opt;
15 |     /* Описание опции, которое предоставляет плагин. */
16 |     const char *opt_descr;
17 | };
18 |
19 | /*
20 |  * Структура, содержащая информацию о плагине.
21 |  */
22 | struct plugin_info {
23 |     /* Назначение плагина */
24 |     const char *plugin_purpose;
25 |     /* Автор плагина, например "Иванов Иван Иванович, N32xx" */
26 |     const char *plugin_author;
27 |     /* Длина списка опций */
28 |     size_t sup_opts_len;
29 |     /* Список опций, поддерживаемых плагином */
30 |     struct plugin_option *sup_opts;
31 | };
32 |
33 |
34 | int plugin_get_info(struct plugin_info* ppi);
35 | /*
36 |  * plugin_get_info()
37 |  *
38 |  * Функция, позволяющая получить информацию о плагине.
39 |  *
40 |  * Аргументы:
41 |  *   ppi - адрес структуры, которую заполняет информацией плагин.
42 |  *
43 |  * Возвращаемое значение:
44 |  *   0 - в случае успеха,
45 |  *   < 0 - в случае неудачи (в этом случае продолжать работу с этим плагином нельзя).
46 |  */
47 |
48 |
49 |
50 | int plugin_process_file(const char *fname,
51 |     struct option in_opts[],
52 |     size_t in_opts_len);
53 | /*
54 |  * plugin_process_file()
55 |  *
56 |  * Функция, позволяющая выяснить, отвечает ли файл заданным критериям.
57 |  *
58 |  * Аргументы:
59 |  *   fname - путь к файлу (полный или относительный), который проверяется на
60 |  *   соответствие критериям, заданным с помощью массива in_opts.
61 |  *
62 |  *   in_opts - список опций (критериев поиска), которые передаются плагину.
63 |  *   struct option {
64 |  *       const char *name;
65 |  *       int has_arg;
66 |  *       int *flag;
67 |  *       int val;
68 |  *   };
69 |  *   Поле name используется для передачи имени опции, поле flag - для передачи
70 |  *   значения опции (в виде строки). Если у опции есть аргумент, поле has_arg
71 |  *   устанавливается в ненулевое значение. Поле val не используется.
```

```
72 |  
73 |         in_opts_len - длина списка опций.  
74 |  
75 |     Возвращаемое значение:  
76 |         0 - файл отвечает заданным критериям,  
77 |         > 0 - файл НЕ отвечает заданным критериям,  
78 |         < 0 - в процессе работы возникла ошибка  
79 |  
80 |     В случае, если произошла ошибка, переменная errno должна устанавливаться  
81 |     в соответствующее значение.  
82 | */  
83 |  
84 | #endif  
85 |
```